OpenJML e Introduzione agli assert di Java

Elvinia Riccobene - Valentina Ciriani

OpenJML

Esempi ed esercizio

Esempi iniziali con metodi statici:

Examples.java

SumProductExamples.java

Esempio semplice di classe con attributo e metodo puro:

Counter.java

Esempio di classe con invariante:

PositiveCounterWithInvariant.java

Esercizio

Sokoban partendo da Sokoban.java sul sito (esercizioSokobanJML.pdf)

Assert java per il design-by-contract

Assert

•Un assert è una formula booleana che consente di verificare se le proprie ipotesi, su un pezzo di codice, sono esatte

• Un assert contiene un'espressione booleana che si ritiene essere vera quando l'assert viene eseguito

• Se non è vera, viene generato un errore

Assert

•Un assert è una dichiarazione di una formula booleana che il programmatore pensa che debba valere in un certo punto di un programma

•Un assert non dovrebbe modificare la logica di un programma

• Un assert contiene un'espressione booleana che si ritiene essere vera quando l'assert viene eseguito

•Se un assert è falso allora

- c'è un bug nel programma
- e viene generato un errore (eccezione)

Assert

- Esempio:

```
d = a*a;
assert d >= 0;
```

Sintassi

L'**assert** ha due forme
La prima forma è:

```
assert Expressione1;
```

dove `Expressione1` è un'espressione booleana

Esecuzione dell'assert:
`Expressione1` è valutata: se è false genera un AssertionError senza nessun messaggio dettagliato

Sintassi

Il secondo tipo di assert è:

```
assert Expressione1 : Expressione2;
```

dove:

`Expressione1` è un'espressione booleana

`Expressione2` è un'espressione che ha un valore

Il sistema passa il valore di `Expressione2` al costruttore AssertionError, che usa la sua rappresentazione in stringa come messaggio di errore

Side effect

- Le espressioni nelle asserzioni devono essere prive di side effect:
  - la valutazione dell'espressione non deve influire su alcuno stato visibile al termine della valutazione
- Eccezione: le asserzioni possono modificare lo stato utilizzato solo dall'interno di altre asserzioni
- Esempio:

```
assert persona.remove();
```

- È corretto solo con assert abilitate

```
boolean removed = persona.remove();
assert removed;
```

- Corretto sempre

Uso degli assert

```
if (i % 3 == 0) {  ...
} else if (i % 3 == 1) {  ...
} else { // Caso in cui (i % 3 == 2)
    ...
}

if (i % 3 == 0) {  ...
} else if (i % 3 == 1) { ...
} else {
    assert i % 3 == 2 : i;
    ...
}
```

Assert per il controllo del flusso

```
void foo() {
    for (...) {
      if (...)
        return;
    }
    // L'esecuzione non dovrebbe mai arrivare qui
}

void foo() {
    for (...) {
      if (...)
        return;
    }
    assert false;
}
```

Precondizioni su metodi public

- Per convenzione: le precondizioni sui metodi `public` vengono implementate da controlli espliciti che generano particolari eccezioni
- Esempio:

```
public void setRefreshRate(int rate) {
  if (rate <= 0 || rate > MAX_REFRESH_RATE)
    throw new IllegalArgumentException("Illegal rate:
" + rate);
    setRefreshInterval(1000/rate);
}
```

- Non si usano gli assert per verificare le precondizioni sui metodi `public`

Precondizioni su metodi non public

- Precondizione
  - su un metodo non pubblico
  - che vale indipendentemente dall'uso della classe
- Esempio:

```
/**
  * Sets the refresh interval (must correspond to a legal frame
  * rate).
  *
  * @param  interval refresh interval in milliseconds.
  */
  private void setRefreshInterval(int interval) {
      // Precondizione in un metodo non public
      // prima istruzione del metodo
      assert interval > 0 && interval <= 1000/MAX_REFRESH_RATE;
      …
  }
```

Postcondizioni

- Sia in metodi pubblici che non pubblici
- Esempio

```
public static int getTrueIndex(boolean[] arrBool) {
      for(int i = 0; i < arrBool.length; i++) {
         if(arrBool[i]) {
            assert arrBool[i];
            return i;
         }
      }
      for(int i = 0; i < arrBool.length; i++){
             assert !arrBool[i];
      }
      return -1;
   }
```

Postcondizioni

A volte ci dobbiamo ricordare il vecchio valore di una variabile (come \old() in JML)

```
//@ ensures counter == \old(counter)-1;
  public void decr() {
     int oldcounter = counter;
     counter--;
     assert counter == oldcounter-1;
  }
```

Invarianti di classe

Gli invarianti di classe devono valere:

- dopo i costruttori,
- prima e dopo l'esecuzione di ogni metodo della classe

Esempio: classe con un attributo `counter`

```
private boolean counterPositivo() {
  return counter > 0;
}
```

Ogni costruttore e ogni metodo della classe deve contenere prima del `return` (e ogni metodo deve contenere come prima istruzione):

```
assert counterPositivo();
```

assert java vs assert JML

- Anche JML ha la parola chiave `assert`, ma è più espressivo che in java
- Esempio: se voglio richiedere che un vettore a non contenga null, in Java:

```
for (n = 0; n < a.length; n++)
assert(a[n] != null);
```

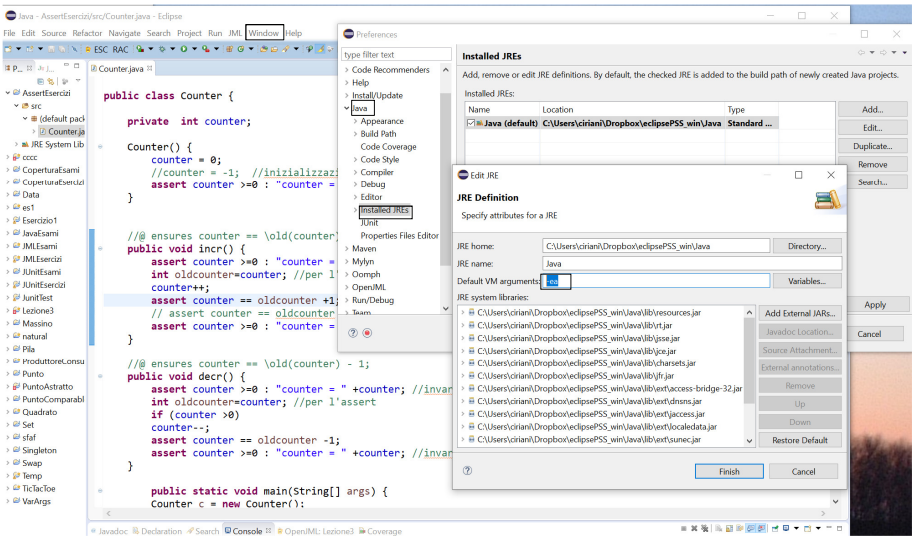
- In JML (all'interno di un codice):

```
/*@ assert
(\forall int i; 0<=i && i<n ;a[i] !=
  null);@*/
```

assert ed eccezioni

- Gli assert hanno lo scopo di rilevare gli errori di programmazione, ovvero servono a verificare situazioni che non dovrebbe accadere
- Un assert è un meccanismo per verificare che un determinato predicato sia vero in un certo punto del codice, se il predicato è falso viene chiamata un'eccezione
- Un'eccezione verifica qualcosa che può o potrebbe accadere cioè una condizione anomala (un evento eccezionale)

Abilitare gli assert in eclipse



Esempio

CounterAssert.java

Esercizi per casa

Risolvere (annotazioni JML) gli esercizi di esame:

- Gioco dell'8 (esercizioGiocoOttoJML.pdf)
- Knight's tour (esercizioKnightTourJML.pdf)